Werkzeuge · Warm-up-Karten

# Warm-up-Karten Welt 0 + Welt 1

Kurze Papier-Aufgaben für den Einstieg jeder Doppelstunde („10' Papier-Warm-up, ohne Gerät"). Jede Karte ist in 5–8 Minuten handschriftlich lösbar und trainiert genau die Formate, die im Abitur verlangt werden – Struktogramme, Tracetabellen, Klassenkarten, Objektkarten und Java von Hand.

### So funktioniert's

- Karte ziehen (oder zugeteilt bekommen), **ohne Gerät** auf einem eigenen Blatt lösen.
- Selbst- oder Partnerkontrolle mit dem Lösungsteil (gleiche Nummer). Ohne Punkte – aber ehrlich.
- Nachüben im **Warm-up-Trainer** des Algo Quest Lab (Karteikasten, ohne Punkte); 30 Karten in Fach 5 → Abzeichen **Warm-up-Meister**.
- Die gleichen Karten stehen digital im Warm-up-Trainer des Lab (Datei warmup_w0_w1.json).

**Achtung Abi:**
Im Abitur schreibst du alles von Hand. Achte auf die Abi-Check-Liste: Signatur wie gefordert · Rückgabe in jedem Pfad · Leer-/Index-/null-Abfrage vor jedem Zugriff · Schleifengrenzen 0 bis länge − 1 · Zeichenketten mit equals vergleichen.

## Kartenübersicht

| Nr. | Kategorie | Titel | Min. | Einsatz |
|---|---|---|---|---|
| W0-01 | Struktogramm lesen | Das Rätsel des Portalwächters | 5 | DS 2 |
| W0-02 | Code ↔ Struktogramm | Vom Code zum Bauplan | 6 | DS 2 |
| W0-03 | Tracetabelle | Spurensuche im Portal | 6 | DS 2 |
| W0-04 | Fehler im Struktogramm | Der kaputte Bauplan | 6 | DS 2 |
| W0-05 | Fehler im Java-Code | Rückwärts durchs Portal | 6 | DS 3 |
| W0-06 | Lückencode | Der Spiegelzauber | 5 | DS 3 |
| W0-07 | Methode implementieren | Die Spannweite des Portals | 7 | DS 5 |
| W0-08 | Funktionsweise erläutern | Das Zählwerk des Portals | 5 | DS 3 |
| W1-01 | Klasse vs. Objekt | Bauplan oder Held? | 5 | DS 9 |
| W1-02 | Bestandteile einer Klasse | Anatomie eines Monsters | 6 | DS 9 |
| W1-03 | Objektkarten und Referenzen | Zwei Namen, ein Held | 6 | DS 10 |
| W1-04 | Objektkarten und Referenzen | Was steckt im Rucksack? | 7 | DS 11 |
| W1-05 | Klassendiagramm | Vom Code zur Klassenkarte | 5 | DS 10 |
| W1-06 | Klassendiagramm | Der Stammbaum der Gegenstände | 8 | Boss-Training |
| W1-07 | Lückencode | Die Schmiede der Waffen | 5 | Boss-Training |
| W1-08 | Lückencode | Ab in den Rucksack | 5 | DS 11 |
| W1-09 | Fehler im Java-Code | Der falsche Spürsinn | 6 | DS 11 |
| W1-10 | Fehler im Java-Code | Der gierige Drache | 6 | DS 11 |
| W1-11 | Methode implementieren | Angriff und Auslese | 6 | DS 11 |
| W1-12 | Methode implementieren | Weg damit! | 8 | DS 11 |

| Nr. | Kategorie | Titel | Min. | Einsatz |
|---|---|---|---|---|
| W1-13 | Verändern und Erweitern | Die Rüstung des Helden | 7 | DS 9 |
| W1-14 | Funktionsweise erläutern | Die Aufräum-Magie | 6 | Boss-Training |
| W1-15 | Funktionsweise erläutern | Welche beschreibung() läuft? | 6 | Boss-Training |

Druckhinweis: Kartenseiten einseitig drucken, an den gestrichelten Linien schneiden (2 Karten im Format A5 je Seite), ggf. laminieren. Der Lösungsteil ist für die Kontrollstation bzw. das Lehrerexemplar gedacht.

WARM-UP W0-01 **Struktogramm lesen** Welt 0 – Die Akademie · ⏱ 5 min

## Das Rätsel des Portalwächters

Der Portalwächter prüft Zahlenfolgen mit der Operation geheim.

**a)** Gib den Rückgabewert für r = [3, 7, 2, 9, 9, 4] an. Notiere dazu nach jedem Schleifendurchlauf die Werte von i und z.

**b)** Beschreibe in einem Satz, was die Operation allgemein berechnet.

**c)** Gib eine Reihung der Länge 5 an, für die der Rückgabewert so groß wie möglich ist.

```
geheim(r: Reihung von Ganzzahl): Ganzzahl
  z ← 0
  für i von 1 bis länge(r) − 1
    falls r[i] > r[i − 1]
      dann: z ← z + 1
      sonst: —
  gib z zurück
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W0-01

WARM-UP W0-02 **Code ↔ Struktogramm** Welt 0 – Die Akademie · ⏱ 6 min

## Vom Code zum Bauplan

Übersetze die Java-Methode in ein Struktogramm nach Abi-Notation:

Kopf mit Signatur (deutsche Typnamen), Zuweisung mit ←, Zählschleife, Verzweigung, „gib … zurück“.

```
public static int zaehleGerade(int[] r) {
    int anzahl = 0;
    for (int i = 0; i < r.length; i++) {
        if (r[i] % 2 == 0) {
            anzahl = anzahl + 1;
        }
    }
    return anzahl;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W0-02

WARM-UP W0-03 Tracetabelle

Welt 0 – Die Akademie · ⏱ 6 min

## Spurensuche im Portal

**a)** Erstelle eine Tracetabelle für den Aufruf mystery(r) mit r = [5, 8, 3, 6, 1, 4].

Spalten: i | r[i] | a | b – eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle (erste Zeile: Werte vor der Schleife).

**b)** Gib den Rückgabewert an und beschreibe, was die Methode allgemein berechnet.

```
public static int mystery(int[] r) {
    int a = r[0];
    int b = 0;   // → Tabellenzeile (Startwerte)
    for (int i = 1; i < r.length; i++) {
        if (r[i] < a) {
            a = r[i];
            b = i;
        }   // → Tabellenzeile
    }
    return b;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W0-03

WARM-UP W0-04 Fehler im Struktogramm

Welt 0 – Die Akademie · ⏱ 6 min

## Der kaputte Bauplan

Das Struktogramm soll das größte Element einer nicht leeren Reihung zurückgeben. Es enthält drei Fehler.

**a)** Finde die drei Fehler und erläutere jeweils ihre Auswirkung.

**b)** Gib ein korrigiertes Struktogramm an.

```
maximum(r: Reihung von Ganzzahl): Ganzzahl
  // r ist nicht leer
  max ← 0
  für i von 1 bis länge(r)
    falls r[i] > max
      dann: max ← r[i]
            gib max zurück
      sonst: –
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W0-04

WARM-UP W0-05 Fehler im Java-Code

Welt 0 – Die Akademie · ⏱ 6 min

## Rückwärts durchs Portal

Die Methode soll den Index des letzten Vorkommens von x in r liefern. Kommt x nicht vor, soll −1 geliefert werden.

**a)** Finde die drei Fehler und erläutere jeweils ihre Auswirkung.

**b)** Gib eine korrigierte Fassung an.

```
public static int letzterIndexVon(int[] r, int x) {
    int pos = 0;
    for (int i = r.length; i > 0; i--) {
        if (r[i] == x) {
            return i;
        }
    }
    return pos;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W0-05

WARM-UP W0-06 Lückencode

Welt 0 – Die Akademie · ⏱ 5 min

## Der Spiegelzauber

Die Methode kehrt die Reihenfolge der Elemente in r um, ohne eine neue Reihung anzulegen.

Beispiel: [1, 2, 3, 4, 5] wird zu [5, 4, 3, 2, 1].

**a)** Ergänze die Lücken (1) bis (4).

**b)** Begründe, warum die Schleife nur bis zur Mitte der Reihung laufen darf.

```
public static void umdrehen(int[] r) {
    for (int i = 0; i < ___(1)___; i++) {
        int tmp = ___(2)___;
        r[i] = r[___(3)___];
        r[r.length - 1 - i] = ___(4)___;
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W0-06

WARM-UP W0-07 Methode implementieren

Welt 0 – Die Akademie · ⏱ 7 min

## Die Spannweite des Portals

Implementiere die Methode spannweite(r: Reihung von Ganzzahl): Ganzzahl.

Sie liefert die Differenz zwischen dem größten und dem kleinsten Element. Für eine leere Reihung wird 0 geliefert.

Beispiele: [4, 9, 1, 7] → 8 · [5] → 0 · [] → 0

Tipp: Ein einziger Schleifendurchlauf genügt. Denk an die Leer-Abfrage vor dem ersten Zugriff!

```
public static int spannweite(int[] r) {
    // dein Code
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W0-07

WARM-UP W0-08 Funktionsweise erläutern

Welt 0 – Die Akademie · ⏱ 5 min

## Das Zählwerk des Portals

**a)** Gib die zurückgegebene Reihung für r = [2, 5, 1, 4] an.

**b)** Erläutere die Funktionsweise der Methode. Gehe dabei auf die Rolle der Variablen summe ein.

**c)** Begründe, ob r nach dem Aufruf verändert ist.

```
public static int[] kumuliert(int[] r) {
    int[] erg = new int[r.length];
    int summe = 0;
    for (int i = 0; i < r.length; i++) {
        summe = summe + r[i];
        erg[i] = summe;
    }
    return erg;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W0-08

WARM-UP W1-01 **Klasse vs. Objekt**

Welt 1 – Heldenschmiede · ⏱ 5 min

## Bauplan oder Held?

**a)** Erläutere den Unterschied zwischen einer Klasse und einem Objekt am Beispiel der Heldenschmiede.

**b)** Gib an, wie viele Klassen im Programmausschnitt verwendet und wie viele Objekte erzeugt werden. Begründe.

**c)** Gib die Lebenspunkte von aria, bruno und goblin nach Ausführung des Ausschnitts an.

```
Held aria = new Held("Aria");
Held bruno = new Held("Bruno", 120, 14);
Monster goblin = new Monster("Goblin", 30, 6);
Held h = aria;
h.nimmSchaden(25);
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-01

WARM-UP W1-02 **Bestandteile einer Klasse**

Welt 1 – Heldenschmiede · ⏱ 6 min

## Anatomie eines Monsters

**a)** Benenne Attribute (mit Typ), Konstruktor und Methoden der Klasse Monster. Gib je Methode an, ob sie sondierend (liefert nur einen Wert) oder verändernd ist.

**b)** Erläutere die Bedeutung von private und public in dieser Klasse.

**c)** Erkläre, wozu im Konstruktor this benötigt wird.

```
public class Monster {
    private String name;
    private int leben;
    private int staerke;
    public Monster(String name, int leben, int staerke) {
        this.name = name;
        this.leben = leben;
        this.staerke = staerke;
    }
    public int getLeben() { return leben; }
    public void nimmSchaden(int schaden) {
        leben = leben - schaden;
        if (leben < 0) { leben = 0; }
    }
    public boolean istBesiegt() { return leben <= 0; }
    public void angreifen(Held h) { h.nimmSchaden(staerke); }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-02

WARM-UP W1-03 Objektkarten und Referenzen

Welt 1 – Heldenschmiede · ⏱ 6 min

## Zwei Namen, ein Held

**a)** Zeichne die Objektkarten nach Ausführung des Ausschnitts (nur name, leben, gold) und die Referenzpfeile der Variablen a, b und c.

**b)** Gib den Wert der Ausdrücke an:

a == c · a.getName().equals(c.getName()) · a.getLeben() · c.getGold()

```
Held a = new Held("Aria");
Held b = a;
Held c = new Held("Aria");
b.nimmSchaden(30);
c.erhalteGold(15);
a.heilen(10);
b = c;
b.erhalteGold(-5);
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-03

WARM-UP W1-04 Objektkarten und Referenzen

Welt 1 – Heldenschmiede · ⏱ 7 min

## Was steckt im Rucksack?

**a)** Zeichne die Objektkarte von h (nur inventar, anzahl, waffe) sowie alle Objekte, auf die h verweist. Stelle die Reihung inventar mit allen 5 Feldern dar.

**b)** Gib an, wie viele Gegenstand-Objekte existieren und welchen Wert h.gesamtWert() liefert.

**c)** Gib die Rückgabewerte der vier Aufrufe von aufnehmen an.

```
Held h = new Held("Aria");
Gegenstand f = new Gegenstand("Fackel", 5);
h.aufnehmen(f);
h.aufnehmen(new Trank("Heiltrank", 20, 30));
h.aufnehmen(null);
h.aufnehmen(f);
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-04

**WARM-UP W1-05** Klassendiagramm

Welt 1 – Heldenschmiede · ⏱ 5 min

## Vom Code zur Klassenkarte

**a)** Erstelle zur Java-Klasse Gegenstand die Klassenkarte nach Abi-Konventionen (Sichtbarkeit + / -, deutsche Typnamen, Konstruktor mit c).

**b)** Begründe, ob der Wert eines Gegenstands nach der Erzeugung noch verändert werden kann.

```
public class Gegenstand {
    private String name;
    private int wert;

    public Gegenstand(String name, int wert) {
        this.name = name;
        this.wert = wert;
    }

    public String getName() { return name; }
    public int getWert() { return wert; }

    public String beschreibung() {
        return name + " (" + wert + " Gold)";
    }
}
```

**PAPIER** ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-05

**WARM-UP W1-06** Klassendiagramm

Welt 1 – Heldenschmiede · ⏱ 8 min

## Der Stammbaum der Gegenstände

**a)** Zeichne ein Klassendiagramm mit den Klassen Held, Gegenstand, Waffe, Trank und Schluessel. Gib in jeder Klassenkarte alle Attribute an; Methoden darfst du durch „…“ ersetzen. Stelle alle Vererbungs- und Assoziationsbeziehungen nach Abi-Konventionen dar.

**b)** Erläutere, warum Held zwei Assoziationen zu Gegenstand-Klassen besitzt.

**PAPIER** ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-06

WARM-UP W1-07 Lückencode

Welt 1 – Heldenschmiede · ⏱ 5 min

## Die Schmiede der Waffen

Die Klasse Waffe erbt von Gegenstand. beschreibung() liefert z. B. "Waffe: Schwert (50 Gold), Schaden 12".

**a)** Ergänze die Lücken (1) bis (3).

**b)** Erläutere, warum in beschreibung() nicht einfach name + " (" + wert + … geschrieben werden kann.

```
public class Waffe ___(1)___ Gegenstand {
    private int schaden;

    public Waffe(String name, int wert, int schaden) {
        ___(2)___;
        this.schaden = schaden;
    }

    public int getSchaden() {
        return schaden;
    }

    public String beschreibung() {
        return "Waffe: " + ___(3)___ + ", Schaden " + schaden;
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-07

WARM-UP W1-08 Lückencode

Welt 1 – Heldenschmiede · ⏱ 5 min

## Ab in den Rucksack

Die Methode aufnehmen der Klasse Held legt g am Ende des Inventars ab. Sie liefert false, wenn g null ist oder das Inventar voll ist, sonst true.

**a)** Ergänze die Lücken (1) bis (5).

**b)** Begründe, warum die Reihenfolge der beiden Teilbedingungen in Zeile 2 keine Rolle spielt – im Gegensatz zu g == null || g.getWert() > 100.

```
public boolean aufnehmen(Gegenstand g) {
    if (g == ___(1)___ || anzahl ___(2)___ inventar.length) {
        return false;
    }
    inventar[___(3)___] = g;
    ___(4)___;
    return ___(5)___;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-08

WARM-UP W1-09 Fehler im Java-Code

Welt 1 – Heldenschmiede · ⏱ 6 min

## Der falsche Spürsinn

Die Methode enthaelt(name: Zeichenkette): Wahrheitswert der Klasse Held soll prüfen, ob ein Gegenstand mit dem angegebenen Namen im Inventar liegt. Sie enthält vier Fehler.

**a)** Finde die Fehler und erläutere jeweils ihre Auswirkung.

**b)** Gib eine korrigierte Fassung an.

```
public boolean enthaelt(String name) {
    for (int i = 0; i <= anzahl; i++) {
        if (inventar[i].getName() == name) {
            return true;
        } else {
            return false;
        }
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-09

WARM-UP W1-10 Fehler im Java-Code

Welt 1 – Heldenschmiede · ⏱ 6 min

## Der gierige Drache

Die Methode wertvollsterGegenstand(): Gegenstand der Klasse Held soll den Gegenstand mit dem größten Wert liefern – bei Gleichstand den ersten, bei leerem Inventar null. Sie enthält drei Fehler.

**a)** Finde die Fehler und erläutere jeweils, bei welchem Inventar sie sich auswirken.

**b)** Gib eine korrigierte Fassung an.

```
public Gegenstand wertvollsterGegenstand() {
    Gegenstand bester = inventar[0];
    for (int i = 0; i < inventar.length; i++) {
        if (inventar[i].getWert() >= bester.getWert()) {
            bester = inventar[i];
        }
    }
    return bester;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-10

WARM-UP W1-11 Methode implementieren

Welt 1 – Heldenschmiede · ⏱ 6 min

## Angriff und Auslese

Implementiere in der Klasse Held:

**a)** angriffswert(): Ganzzahl – liefert staerke plus den Schaden der ausgerüsteten Waffe; ohne Waffe (waffe == null) nur staerke.

**b)** anzahlTeurerAls(grenze: Ganzzahl): Ganzzahl – liefert die Anzahl der Gegenstände im Inventar, deren Wert größer als grenze ist.

Beispiel: Inventar Fackel (5), Schwert (50), Heiltrank (20) → anzahlTeurerAls(10) liefert 2.

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-11

WARM-UP W1-12 Methode implementieren

Welt 1 – Heldenschmiede · ⏱ 8 min

## Weg damit!

Implementiere in der Klasse Held die Methode entfernen(index: Ganzzahl): Gegenstand.

- Sie liefert den Gegenstand an der Position index und entfernt ihn aus dem Inventar.
- Die nachfolgenden Gegenstände rücken auf (Reihenfolge bleibt erhalten), das frei gewordene letzte Feld wird null, anzahl sinkt um 1.
- Ist index ungültig (kleiner 0 oder nicht kleiner als anzahl), wird null geliefert; das Inventar bleibt unverändert.

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-12

WARM-UP W1-13 Verändern und Erweitern

Welt 1 – Heldenschmiede · ⏱ 7 min

## Die Rüstung des Helden

Helden sollen eine Rüstung tragen können, die jeden Treffer abschwächt.

**a)** Ergänze in der Klasse Held ein Attribut ruestung (Ganzzahl, Anfangswert 0) und eine Methode setRuestung(r: Ganzzahl): nichts, die nur Werte ≥ 0 übernimmt.

**b)** Verändere nimmSchaden(schaden: Ganzzahl): nichts: Der tatsächliche Schaden ist schaden − ruestung, aber nie kleiner als 0. leben fällt weiterhin nie unter 0.

**c)** Gib an, wie sich nimmSchaden(4) bei ruestung = 6 auswirkt, und begründe, warum der Mindestwert 0 nötig ist.

```
public void nimmSchaden(int schaden) {
    leben = leben - schaden;
    if (leben < 0) {
        leben = 0;
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-13

WARM-UP W1-14 Funktionsweise erläutern

Welt 1 – Heldenschmiede · ⏱ 6 min

## Die Aufräum-Magie

Die folgende Methode der Klasse Held wird aufgerufen. Das Inventar enthält vorher:

Fackel (5), Schwert (50), Seil (3), Heiltrank (20)

**a)** Gib den Inhalt von inventar (alle 5 Felder) und den Wert von anzahl nach dem Aufruf an.

**b)** Erläutere die Funktionsweise der Methode. Gehe auf die Rolle von j ein.

**c)** Gib einen passenden Methodennamen an.

```
public void geheimnis() {
    int j = 0;
    for (int i = 0; i < anzahl; i++) {
        if (inventar[i].getWert() >= 10) {
            inventar[j] = inventar[i];
            j++;
        }
    }
    for (int i = j; i < anzahl; i++) {
        inventar[i] = null;
    }
    anzahl = j;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-14

WARM-UP W1-15 Funktionsweise erläutern



# Welche beschreibung() läuft?

**a)** Gib die Konsolenausgabe des Programmausschnitts an.

**b)** Erläutere, warum für die drei Objekte unterschiedliche Methoden ausgeführt werden, obwohl alle in einer Reihung von Gegenstand liegen.

**c)** Zeile A wird einkommentiert. Begründe, warum der Compiler einen Fehler meldet, und gib eine Korrektur an.

```
Gegenstand[] kiste = new Gegenstand[3];
kiste[0] = new Waffe("Schwert", 50, 12);
kiste[1] = new Schluessel("Rostiger Schlüssel", 2, 7);
kiste[2] = new Trank("Heiltrank", 20, 30);
for (Gegenstand g : kiste) {
    println(g.beschreibung());
}
Gegenstand x = new Waffe("Dolch", 15, 5);
// println(x.getSchaden());      // Zeile A
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W1-15